Faculty of Engineering & Technology,
Sri Sri University, Cuttack.

Keyshield : Keylogger Detection & Prevention Tool

Group -
Ayushman Das - FET-BCD-2024-28-008
Chinmay Ghabne - FET-BCD-2024-28-003
Satyajeet Pradhan - FET-BCD-2024-28-017
Devesh Patnaik - FET-BCD-2024-28-015

Mentor -
Mr. Ayush Joshi

Table of Contents

1. **INTRODUCTION**
2. **ABSTRACT**
3. **OBJECTIVE**
4. **METHODOLOGY**
5. **IMPLEMENTATION**
6. **TECHNICAL TOOLS**
7. **PROGRESS REPORT**
8. **PROJECT TIMELINE**
9. **KEY DECISIONS**
10. **CONCLUSION**

THE PROBLEM

Unlike ransomware that announces its presence, keyloggers operate covertly in the background, recording every keystroke without any visible indication. They harvest credentials, banking details, and sensitive communications over extended periods, making them one of the most insidious forms of malware.

Traditional antivirus solutions rely on signature-based detection, which proves ineffective against zero-day threats or custom-built keyloggers with unique code patterns. This fundamental limitation leaves systems vulnerable to sophisticated, evolving spyware.

Abstract

- **KEYSHIELD** is a proactive endpoint security framework and interface- designed to detect, block, and analyze software and hardware keylogging attempts.
- It monitors system behavior to identify malicious processes attempting to hook into keystroke data or access the clipboard.
- When a potential keylogger is identified, the system:
  - Captures the suspicious process details.
  - Analyzes the API hooking behavior.
  - Alerts the user and isolates the process.
  - Keeps sensitive credentials completely safe.
- The project provides multi-layer protection through signature-based scanning, behavioral analysis, and a secure virtual keyboard environment.
- This helps identify credential harvesting, spyware activity, and unauthorized data exfiltration before real damage occurs.

Objectives

Real-Time Hook Interception System

Wide monitoring of keyboard hook installations using WH_KEYBOARD_LL hooks across all processes and sessions

Behavioral Pattern Analysis

Dynamic threat scoring based on API call frequency, process injection patterns, and timing anomalies rather than static signatures

Automated Mitigation

Automatic unhooking of malicious functions, process termination, and quarantine procedures when threat thresholds are exceeded

Methodology

Kernel Layer: The Observer Windows Driver Framework (KMDF) implementation monitoring I/O Request Packets (IRPs), tracking process creation events, and intercepting system call tables. Runs at Ring 0 for maximum visibility.

User Layer: The Analyst Background service monitoring API calls including SetWindowsHookEx, GetAsyncKeyState, and RegisterHotKey. Implements behavioral scoring engine and maintains application whitelisting database.

Inter-Layer Communication Device: I/O Control (IOCTL) interface enables secure data exchange between user-mode service and kernel driver using shared memory regions.

Implementation

Hook Detection Engine

- Monitor WH_KEYBOARD_LL hook installations system-wide
- Track high-frequency polling patterns (>1000Hz)
- Detect DLL injection into explorer.exe and other critical processes Analyze keyboard message queue manipulation

Whitelisting System

Automatically exclude digitally signed applications from Microsoft, Valve, and other trusted vendors to minimize false positives while maintaining security posture.

Technology Stack & Development Environment

Core Programming

C++ for kernel driver and behavioral analysis engine. HYBRID set of languages for GUI dashboard and for testing automation scripts.

Development Tools

Visual Studio Code 2022 with Windows Driver Kit (WDK) integration. WinDbg for kernel debugging and crash dump analysis.

Testing Environment

VMware Workstation for isolated malware testing. Separate Windows 10/11 and linux based VMs for kernel mode testing and stability verification.

Progress Report

1. System Architecture Design

A comprehensive system architecture has been conceptualized, focusing on structured user-kernel communication protocols, efficient memory management strategies, and robust inter-process synchronization mechanisms. The detailed design blueprint has been planned and is ready for implementation.

2. Driver Development Environment

The development environment setup has been planned, including configuring Visual Studio 2022 with the Windows Driver Kit (WDK) toolchain. The initial step will involve building and testing a basic “Hello World” kernel driver within a VMware environment to validate the setup before progressing to advanced driver functionalities.

3. API Monitoring Framework

A framework for API monitoring has been proposed, with key Windows APIs for keyboard hooking identified — such as SetWindowsHookEx, CallNextHookEx, and GetAsyncKeyState. Their parameter structures and functional roles have been studied to support the upcoming implementation phase.

Project Timeline

Phase 1: Foundation (January-February)

Architecture design and environment setup completed

Phase 2: Core Engine (March)

Hook detection engine and kernel driver implementation

Phase 3: Interface - MARCH

GUI dashboard development and notification system integration

Phase 4: Validation - APRIL

Malware stress testing and optimization

Key Design Decisions

Kernel Mode Selection

Operating in Ring 0 provides systemlevel authority to monitor all process activities invisible to standard task managers. Kernel drivers can intercept hardware interrupts and modify system call tables, enabling real-time hook interception.

Behavioral Score Approach

Instead of binary allow/deny decisions, the system assigns threat scores based on observed patterns. This reduces false positives while maintaining detection sensitivity for novel threats.

Conclusion & Future Work

Open Challenges

- Rootkit evasion techniques are to be researched
- Performance overhead optimization
- Multi-platform support

Next Steps

- Complete hook detection engine
- Develop real-time notification system
- Execute malware test suite

Thank You for your time !